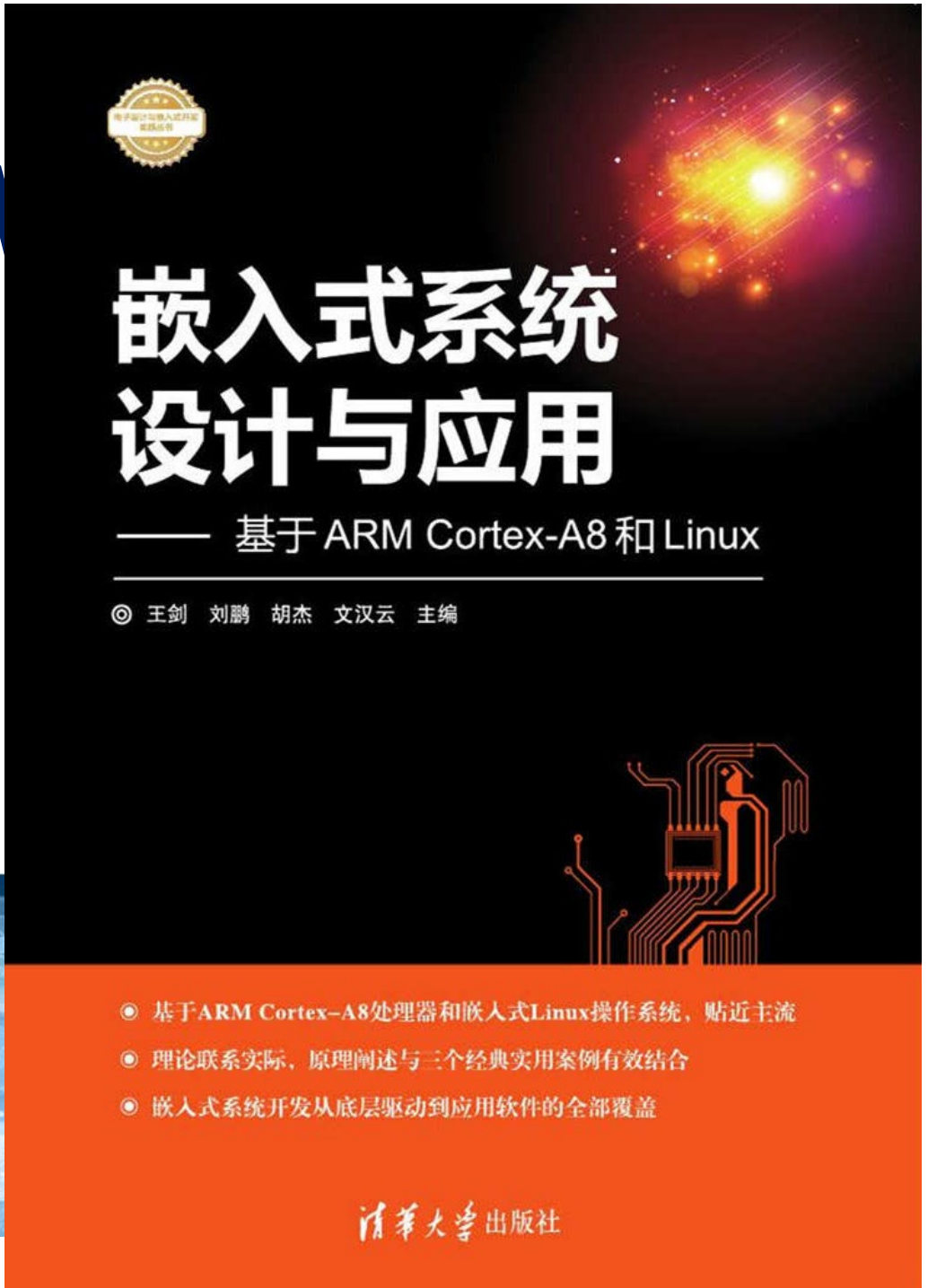
第5章 ARM


嵌入式系统
设计与应用
—— 基于ARM Cortex-A8和Linux
王剑 刘鹏 胡杰 文汉云 主编
基于ARM Cortex-A8处理器和嵌入式Linux操作系统，贴近主流
理论联系实际，原理阐述与三个经典实用案例有效结合
嵌入式系统开发从底层驱动到应用软件的全部覆盖
清华大学出版社

CONTENS

5.1 ARM-Linux概述
5.2 ARM-Linux进程管理
5.3 ARM-Linux内存管理
//5.4 ARM-Linux模块
5.5 ARM-Linux中断管理
5.6 ARM-Linux系统调用

Linux操作系统诞生于1991年，可安装在各种计算机硬件设备中，比如各种智能移动终端、路由器、台式计算机、大型机和超级计算机等。Linux支持包括x86、ARM、MIPS和POWER-PC等在内的多种硬件体系结构。

Linux存在着许多不同的版本，但它们都使用了Linux内核。

Linux是一个一体化内核（monolithic kernel）系统。

这里的“内核”指的是一个提供硬件抽象层、磁盘及文件系统控制、多任务等功能的系统软件，一个内核不是一套完整的操作系统。一套建立在Linux内核的完整操作系统叫作Linux操作系统，或是GNU/Linux。

Linux操作系统的灵魂是Linux内核

内核为系统其他部分提供系统服务。ARM-Linux内核是专门适应ARM体系结构设计的Linux内核，它负责整个系统的：

进程管理和调度、

内存管理、

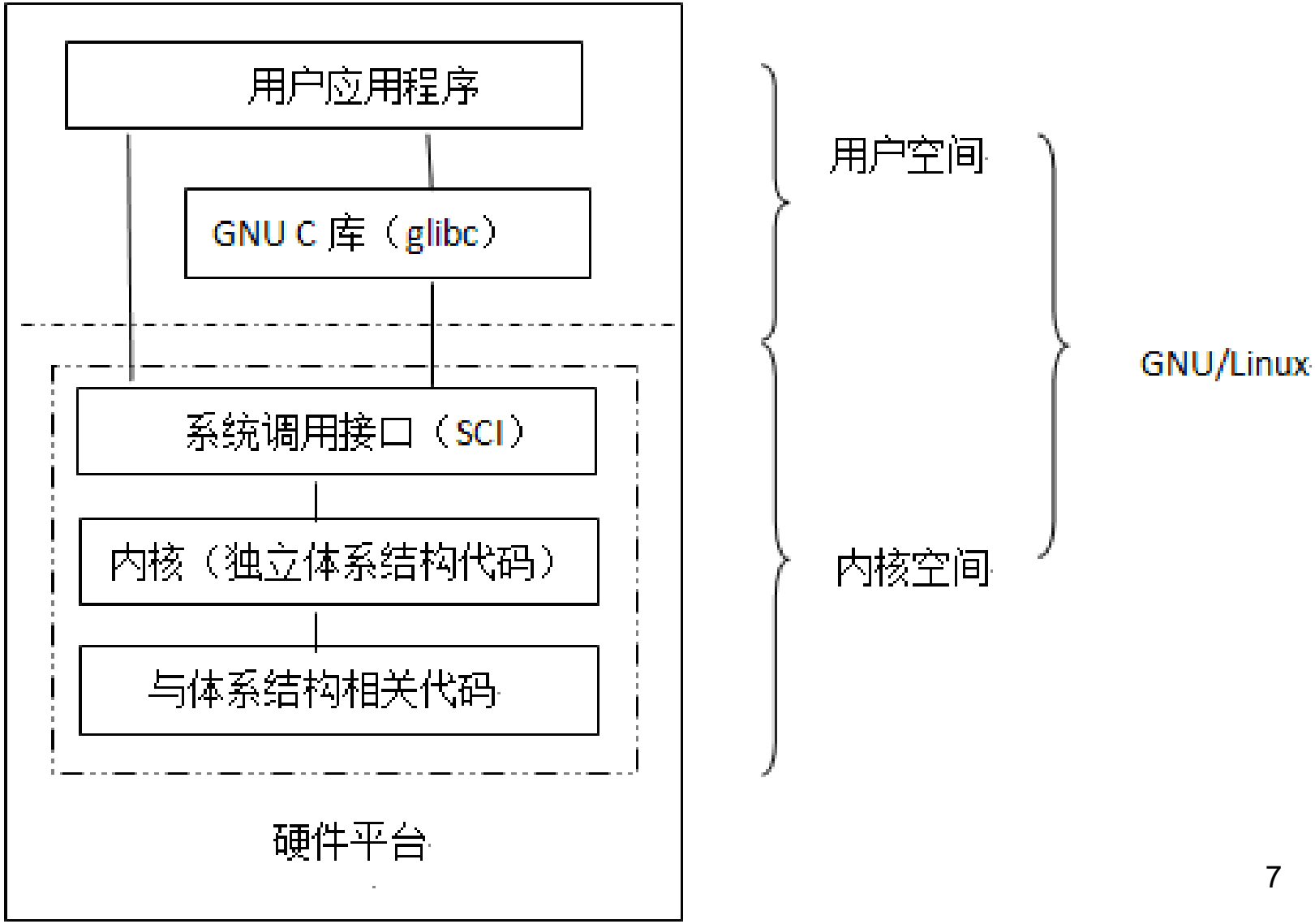
文件管理、

设备管理和网络管理等主要系统功能。

5.1 Part One

ARM-Linux概述

5.1.1 GNU/Linux 操作系统的基本体系结构



用户空间包括用户应用程序和GNU C库（glibc库），负责执行用户应用程序。

内核空间可以进一步划分成 3 层:

系统调用接口（System Call Interface），它是用户空间与内核空间的桥梁

独立于体系结构的内核代码

依赖于体系结构的代码，构成了通常称为板级支持包BSP（Board Support Package）的部分

ARM-Linux内核版本变化与Linux内核版本变化保持同步。

嵌入式linxu系统内核（如ARM_Linux内核）往往在标准linux基础上通过安装patch实现，如ARM_Linux内核就是对linux安装rmk补丁形成的，只有安装了这些补丁，内核才能顺利地移植到ARM_Linux上。

当然也可以通过已经安装好补丁的内核源码包实现。

在2.6版本之前，Linux内核版本的命名格式为 “A.B.C”。

数字 A 是内核版本号，

数字 B 是内核主版本号，主版本号根据传统的奇-偶系统版本编号来分配：奇数为开发版，偶数为稳定版；

数字 C 是内核次版本号，次版本号是无论在内核增加安全补丁、修复bug、实现新的特性或者驱动时都会改变。

2011年5月29号，设计者Linus Torvalds宣布为了纪念Linux发布 20周年，在 2.6.39 版本发布之后，内核版本将升到3.0。Linux继续使用在2.6.0版本引入的基于时间的发布规律，但是使用第二个数——例如在3.0发布的几个月之后发布3.1，同时当需要修复bug和安全漏洞的时候，增加一个数字（现在是第三个数）来表示，如 3.0.18。

Mainline是主线版本，

Stable 是稳定版，由

Longterm是长期支持版

HTTP	https://www.kernel.org/pub/
GIT	https://git.kernel.org/
RSYNC	rsync://rsync.kernel.org/pub/

mainline:	4.3-rc3	2015-09-27	[tar.xz]
stable:	4.2.1	2015-09-21	[tar.xz]
longterm:	4.1.8	2015-09-21	[tar.xz]
longterm:	3.18.21	2015-08-31	[tar.xz]
longterm:	3.14.53	2015-09-21	[tar.xz]
longterm:	3.12.48	2015-09-18	[tar.xz]
longterm:	3.10.89	2015-09-21	[tar.xz]
longterm:	3.4.109	2015-09-18	[tar.xz]
longterm:	3.2.71	2015-08-12	[tar.xz]
longterm:	2.6.32.68	2015-09-18	[tar.xz]
linux-next:	next-20150925	2015-09-25	

Linux是一个内核运行在单独的内核地址空间的***单内核***，但是**汲取了微内核的精华**如模块化设计、抢占式内核、支持内核线程以及动态装载内核模块等特点。

O(1)算法的:一个核心数据结构即为prio_array结构体。

该结构体中有一个用来表示进程动态优先级的数组queue，它包括了每一种优先级进程所形成的链表。

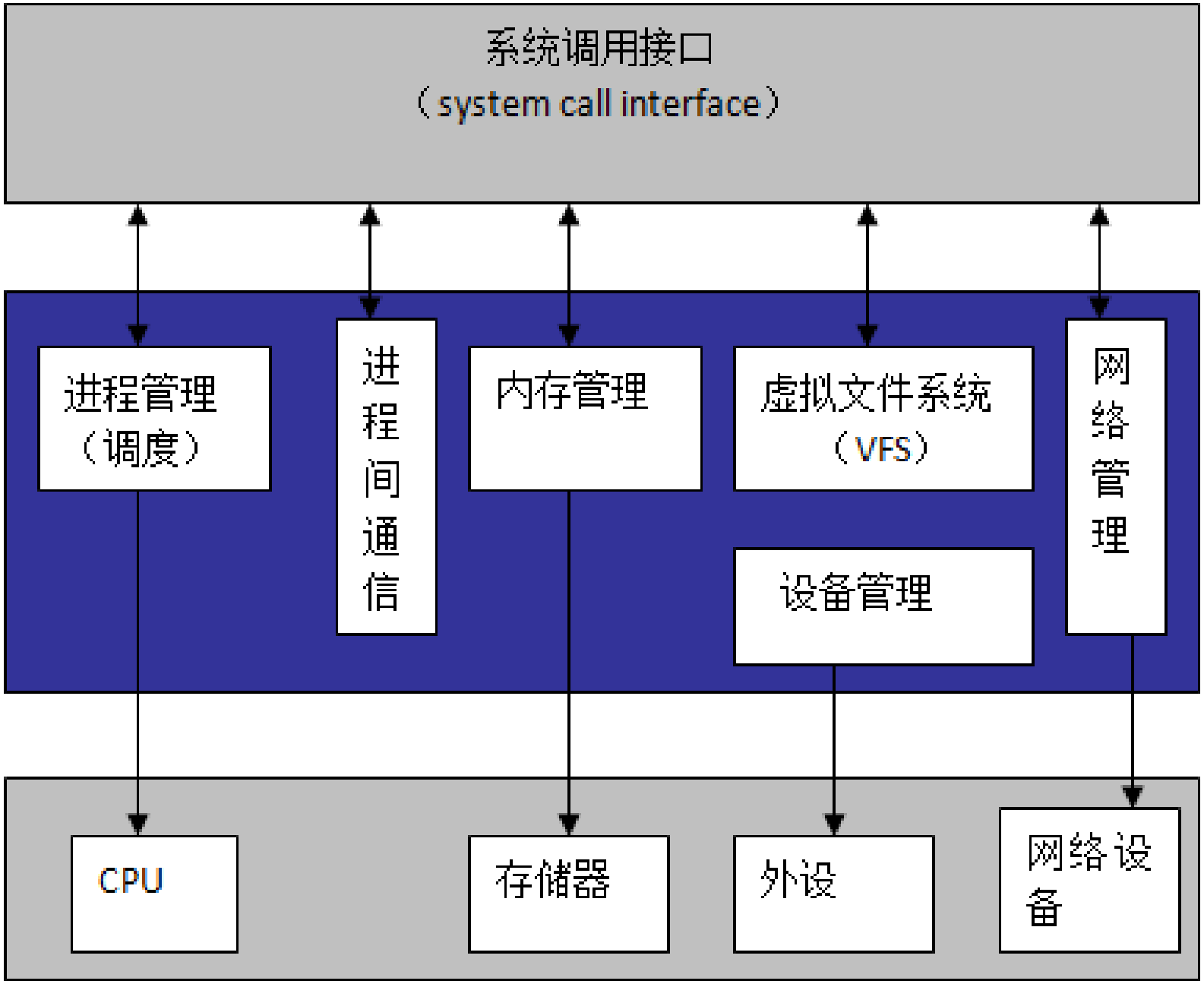
```
#define MAX_USER_RT_PRIO        100
#define MAX_RT_PRIO             MAX_USER_RT_PRIO
#define MAX_PRIO                (MAX_RT_PRIO + 40)
typedef struct prio_array prio_array_t;
struct prio_array {
        unsigned int nr_active;
        unsigned long bitmap[BITMAP_SIZE];
        struct list_head queue[MAX_PRIO];
};
```

因为进程优先级的最大值为139，因此MAX_PRIO的最大值取140（详细的是，普通进程使用100到139的优先级。实时进程使用0到99的优先级）。因此，queue数组中包括140个可执行状态的进程链表，每一条优先级链表上的进程都具有同样的优先级，而不同进程链表上的进程都拥有不同的优先级。

除此之外，prio_array结构中还包含一个优先级位图bitmap。该位图使用一个位（bit）来代表一个优先级，而140个优先级最少须要5个32位来表示，因此BITMAP_SIZE的值取5。起初。该位图中的全部位都被置0，当某个优先级的进程处于可执行状态时。该优先级所相应的位就被置1。

因此，O(1)算法中查找系统最高的优先级就转化成查找优先级位图中第一个被置1的位。

5.1.3 ARM-Linux内核的主要架构及功能



根据内核的核心功能，**Linux**内核具有**5**个主要的子系统，分别负责如下的功能：进程管理、内存管理、虚拟文件系统、进程间通信和网络接口。

1.进程管理

进程管理负责管理**CPU**资源，以便让各个进程能够以尽量公平的方式访问**CPU**。

进程管理负责进程的创建和销毁，并处理它们和外部世界之间的连接(输入输出)。

控制进程如何共享的调度器也是进程管理的一部分。

概括来说，内核进程管理活动就是在单个或多个**CPU**上实现了多个进程的抽象。进程管理源码可参考**./linux/kernel**目录。

2.内存管理

Linux内核所管理的另外一个重要资源是内存。

内存管理策略是决定系统性能好坏的一个关键因素。

内核在有限的可用资源之上为每个进程都创建了一个虚拟空间。内存管理的源代码可以在 **./linux/mm** 中找到。

3.虚拟文件系统

文件系统在Linux 内核中具有十分重要的地位，用于对外设的驱动和存储，隐藏了各种硬件的具体细节。Linux引入了虚拟文件系统（Virtual File System）为用户提供了统一、抽象的文件系统界面，以支持越来越繁杂的具体的文件系统。

Linux内核将不同功能的外部设备，例如Disk设备、输入输出设备、显示设备等，抽象为可以通过统一的文件操作接口来访问。

Linux中的绝大部分对象都可以视为文件并进行相关操作。

4.进程间通信

Linux内核通过支持POSIX规范中标准的IPC（Inter Process Communication，相互通信）机制和其他许多广泛使用的IPC机制实现进程间通信。IPC负责:

比如UNIX中最常见的管道、信号量、消息队列和共享内存等。另外，信号（signal）也常被用来作为进程间的通信手段。Linux内核支持POSIX规范的信号及信号处理并广泛应用.

5.网络管理

网络管理提供了各种网络标准的存取和各种网络硬件的支持，负责管理系统的网络设备，并实现多种多样的网络标准。网络接口可以分为网络设备驱动程序和网络协议。

这五个系统相互依赖，缺一不可
相对而言进程管理处于比较重要的地位，
其他子系统的挂起和恢复进程的运行都必须依靠进程调度子系统的参与。

调度程序的初始化及执行过程中需要内存管理模块分配其内存地址空间并进行处理；

进程间通信需要内存管理实现进程间的内存共享；而内存管理利用虚拟文件系统支持数据交换，交换进程（swapd）定期由调度程序调度；

虚拟文件系统需要使用网络接口实现网络文件系统，而且使用内存管理子系统实现内存设备管理，

同时虚拟文件系统实现了内存管理中内存的交换。

5.2 Part Two

ARM-Linux进程管理

进程是处于执行期的程序以及它所管理的资源的总称，这些资源包括如打开的文件、挂起的信号、进程状态、地址空间等。程序并不是进程，实际上两个或多个进程不仅有可能执行同一程序，而且还有可能共享地址空间等资源。

进程管理是Linux内核中最重要的子系统，它主要提供对CPU的访问控制。由于计算机中，CPU资源是有限的，而众多的应用程序都要使用CPU资源，所以需要“进程调度子系统”对CPU进行调度管理。

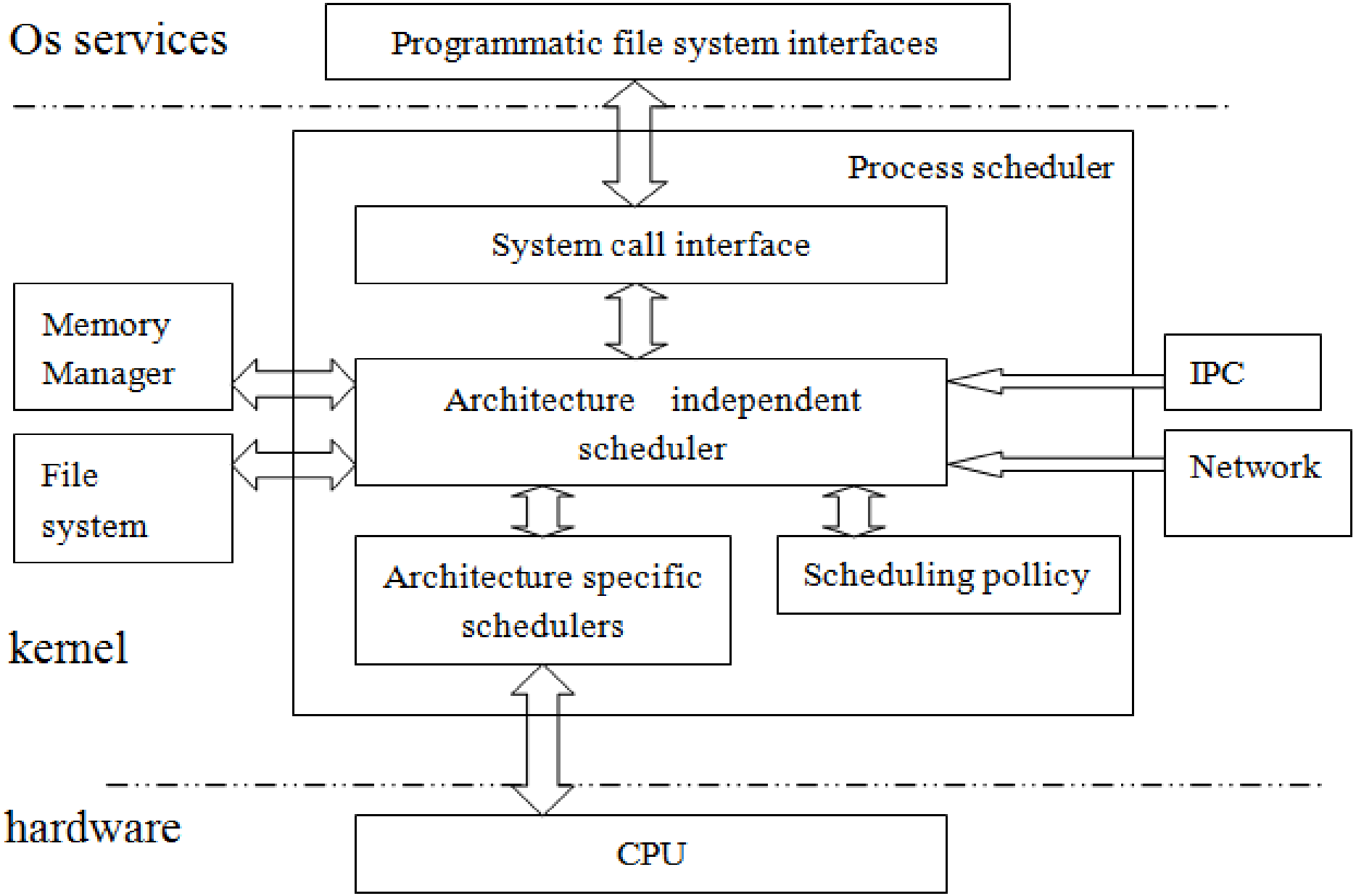


图5-4 Linux进程管理调度子系统基本架构

Scheduling Policy模块。该模块实现进程调度的策略，决定哪个（或者哪几个）进程将拥有CPU资源。

Architecture-specific Schedulers模块。该模块涉及体系结构相关的部分，用于将对不同CPU的控制抽象为统一的接口。这些控制功能主要在suspend和resume进程时使用，包含CPU的寄存器访问、汇编指令操作等。

Architecture-independent Scheduler模块。该模块涉及体系结构无关的部分，会和“Scheduling Policy模块”沟通，决定接下来要执行哪个进程，然后通过“Architecture-specific Schedulers模块”指定的进程予以实现。

System Call Interface，系统调用接口。进程调度子系统通过系统调用接口将需要提供给用户空间的接口开放出去，同时屏蔽掉不需要用户空间程序关心的细节。

5.2.1 进程的表示和切换

Linux内核通过一个被称为进程描述符的task_struct结构体（也叫进程控制块）来管理进程，这个结构体记录了进程的最基本的信息，

进程描述符中不仅包含了许多描述进程属性的字段，而且还包含一系列指向其他数据结构的指针。内核把每个进程的描述符放在一个叫做任务队列的双向循环链表当中，它定义在./include/linux/sched.h文件中。

系统中的每个进程都必然处于以上所列进程状态中的一种。
这里对进程状态给予说明
TASK_RUNNING表示进程要么正在执行，要么正要准备执行。
TASK_INTERRUPTIBLE表示进程被阻塞（睡眠），直到某个条件变为真。条件一旦达成，进程的状态就被设置为TASK_RUNNING。
TASK_UNINTERRUPTIBLE的意义与TASK_INTERRUPTIBLE基本类似，除了不能通过接受一个信号来唤醒以外。
__TASK_STOPPED表示进程被停止执行。
__TASK_TRACED表示进程被debugger等进程监视。
TASK-WAKEKILL该状态是当进程收到致命错误信号时唤醒进程。
TASK_WAKING：该状态说明该任务正在唤醒，其他唤醒操作均会失败。都被置为TASK_DEAD状态。
TASK_DEAD：一个进程在退出时，state字段都被置于该状态。
EXIT_ZOMBIE：表示进程的执行被终止，但是其父进程还没有使用wait()等系统调用来获知它的终止信息。
EXIT_DEAD：进程在系统中被删除时将进入该状态。该状态表示进程的最终状态。
EXIT_ZOMBIE和EXIT_DEAD也可以存放在exit_state成员中。

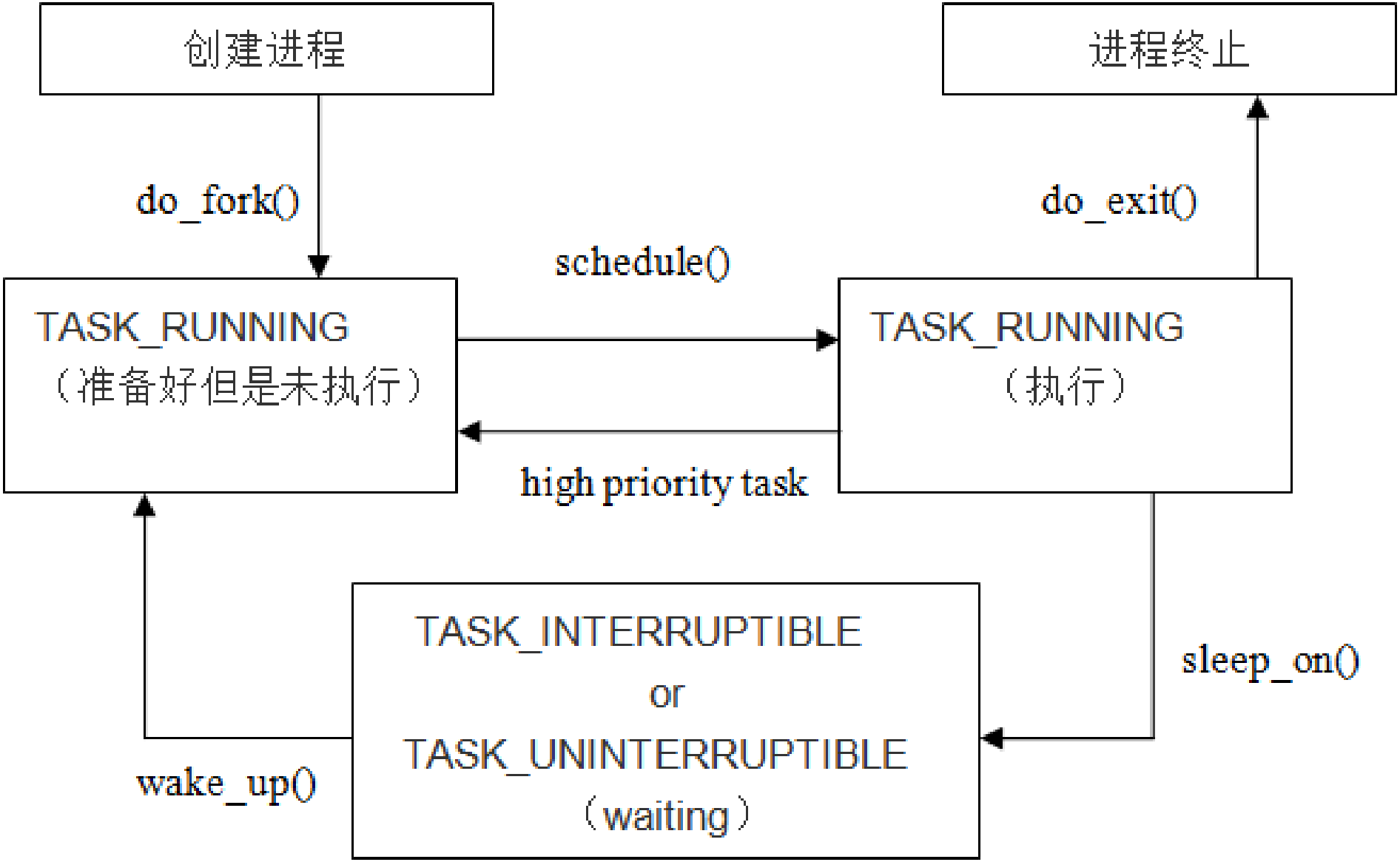

创建进程
进程终止
do_fork()
do_exit()
schedule()
TASK_RUNNING
（准备好但是未执行）
TASK_RUNNING
（执行）
high priority task
TASK_INTERRUPTIBLE
or
TASK_UNINTERRUPTIBLE
（waiting）
sleep_on()
wake_up()

图5-5 进程状态的切换

5.2.2 进程、线程和内核线程

在Linux内核中，内核是采用进程、线程和内核线程统一管理的方法实现进程管理的。

内核将进程、线程和内核线程一视同仁

1.即内核使用唯一的数据结构task_struct来分别表示它们；

2.内核使用相同的调度算法对这三者进行调度；

3.内核也使用同一个函数do_fork()来分别创建这三种执行线程（thread of execution）。

执行线程通常是指任何正在执行的代码实例，比如一个内核线程，一个中断处理程序或一个进入内核的进程。

进程是系统资源分配的基本单位
线程是程序独立运行的基本单位

线程有时候也被称作小型进程

如果内核要对线程进行调度，那么线程必须如同进程那样在内核中对应一个数据结构。进程在内核中有相应的**进程描述符，即task_struct结构**。

在内核中还有一种特殊的线程，称之为*内核线程*（Kernel Thread）。由于在内核中进程和线程不做区分，因此也可以将其称为内核进程。内核线程在内核中也是通过task_struct结构来表示的

内核线程和普通进程的不同:

1.内核线程永远都运行在内核态，而不同进程既可以运行在用户态也可以运行在内核态。从地址空间的使用角度来讲，内核线程只能使用大于3GB的地址空间，而普通进程则可以使用整个4GB的地址空间。

2.

- **内核线程**只能调用内核函数无法使用用户空间的函数
- 普通进程必须通过系统调用才能使用内核函数

5.2.3 进程描述符task_struct的几个特殊字段

（1）mm字段：指向mm_struct结构的指针，该类型用来描述进程整个的虚拟地址空间。其数据结构如下：

```
struct mm_struct *mm，  *active_mm;
#ifdef CONFIG_COMPAT_BRK
    unsigned brk_randomized:1;
#endif
#if defined(SPLIT_RSS_COUNTING)
    struct task_rss_stat    rss_stat;
#endif
```

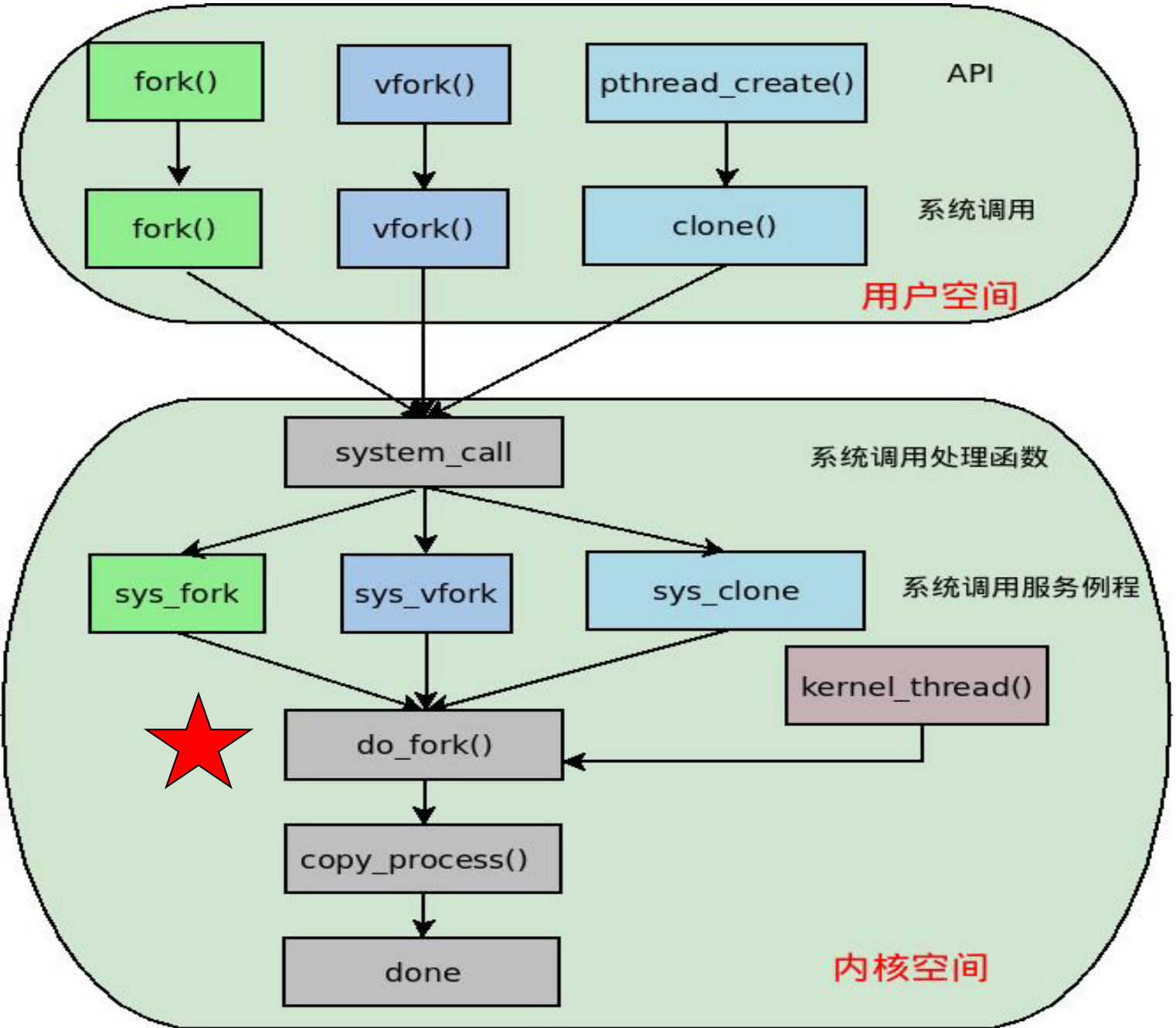
（2）fs字段：指向fs_struct结构的指针，该字段用来描述进程所在文件系统的根目录和当前进程所在的目录信息。

(3）files字段：指向files_struct结构的指针，该字段用来描述当前进程所打开文件的信息。

（4）signal字段：指向signal_struct结构（信号描述符）的指针，该字段用来描述进程所能处理的信号。数据结构:

```
/* signal handlers */
    struct signal_struct *signal;
    struct sighand_struct *sighand;
    sigset_t blocked，  real_blocked;
    sigset_t saved_sigmask; /* restored if set_restore_sigmas
k() was used */
    struct sigpending pending;
    unsigned long sas_ss_sp;
    size_t sas_ss_size;
 int (*notifier)(void *priv);
    void *notifier_data;
    sigset_t *notifier_mask;
```

5.2.4 do_fork()函数



do_fork()函数对于进程、线程以及内核线程的应用

内核中创建进程的核心函数即为do_fork()，该函数的原型如下：

```
long do_fork(unsigned long clone_flags，
             unsigned long stack_start，
             struct pt_regs *regs，
             unsigned long stack_size，
             int __user *parent_tidptr，
             int __user *child_tidptr)
```

该函数的参数的功能说明如下：

clone_flags：代表进程各种特性的标志。低字节指定子进程结束时发送给父进程的信号代码，一般为SIGCHLD信号，剩余三个字节是若干个标志或运算的结果。

stack_start：子进程用户态堆栈的指针，该参数会被赋值给子进程的esp寄存器。

regs：指向通用寄存器值的指针，当进程从用户态切换到内核态时通用寄存器中的值会被保存到内核态堆栈中。

stack_size：未被使用，默认值为0。

parent_tidptr：该子进程的父进程用户态变量的地址，仅当CLONE_PARENT_SETTID被设置时有效。

child_tidptr：该子进程用户态变量的地址，仅当CLONE_CHILD_SETTID被设置时有效。

一个问题：既然进程、线程和内核线程在内核中都是通过do_fork()完成创建的，那么do_fork()是如何体现其功能的多样性？

clone_flags参数在这里起到了关键作用，通过选取不同的标志，从而保证了do_fork()函数实现多角色——创建进程、线程和内核线程——功能的实现。

下面只介绍其中几个主要的标志。

CLONE_VIM：子进程共享父进程内存描述符和所有的页表。

CLONE_FS：子进程共享父进程所在文件系统的根目录和当前工作目录。

CLONE_FILES：子进程共享父进程打开的文件。

CLONE_SIGHAND：子进程共享父进程的信号处理程序、阻塞信号和挂起的信号。使用该标志必须同时设置CLONE_VM标志。

如果创建子进程时设置了上述标志，那么子进程会共享这些标志所代表的父进程资源。

5.2.5 进程的创建

在用户态程序中，可以通过**fork()**、vfork()和**clone()**三个接口函数创建进程，这三个函数在库中分别对应同名的系统调用。系统调用函数通过128号软中断进入内核后，会调用相应的系统调用服务例程。 分别是sys_fork()、sys_vfork()和sys_clone()。

```
int sys_fork(struct pt_regs *regs)
{
        return do_fork(SIGCHLD， regs->sp， regs， 0，
NULL， NULL);
}

int sys_vfork(struct pt_regs *regs)
{
        return do_fork(CLONE_VFORK | CLONE_VM |
SIGCHLD， regs->sp， regs， 0，NULL， NULL);
}
```

```
Long sys_clone(unsigned long clone_flags， unsigned long newsp，
 void __user *parent_tid， void __user *child_tid， struct pt_regs *regs)
 {
        if (!newsp)
                newsp = regs->sp;
        return do_fork(clone_flags， newsp， regs， 0， parent_tid， child_tid);
 }
```

三个系统服务例程内部都调用了*do_fork()*，主要差别在于第一个参数所传的值不同。

5.2.6 线程和内核线程的创建

每个线程在内核中对应一个轻量级进程，两者的关联是通过线程库完成的。因此通过pthread_create()创建的线程最终在内核中是通过clone()完成创建的，而clone()最终调用do_fork()。

一个新内核线程的创建是通过在现有的内核线程中使用kernel_thread()而创建的，其本质也是向do_fork()提供特定的flags标志而创建的。

```
Int kernel_thread(int (*fn)(void*)，void *arg，unsigned long flags)
{
return do_fork(flags|CLONE_VM|CLONE_UNTRACED，0，&regs，0，NULL，NULL);
}
```

从上面的组合的flags标志可以看出，新的内核线程至少会共享父内核线程的内存地址空间。

5.2.7 进程的执行---exec函数族

在linux中中exec函数族提供了一个在进程中启动另一个程序执行的方法。

它可以根据指定的文件名或目录名找到可执行文件，并用它来取代原调用进程的数据段、代码段和堆栈段，在执行完之后，原调用进程的内容除了进程号外，其他全部被新的进程替换了。

实际上，在Linux中并没有exec()函数，
而是有6个以exec开头的函数

所需头文件	#include <unistd.h>
函数原型	int execl(const char *path, const char *arg, ...)
	int execv(const char *path, char *const argv[])
	int execle(const char *path, const char *arg, ..., char *const envp[])
	int execve(const char *path, char *const argv[], char *const envp[])
	int execlp(const char *file, const char *arg, ...)
	int execvp(const char *file, char *const argv[])
函数返回值	-1：出错

5.2.8 进程的终止

当进程终结时，内核必须释放它所占有的资源，并告知其父进程。进程的终止可以通过以下三个事件驱动：

正常的进程结束

信号

exit（）函数的调用。

进程的终结最终都要通过do_exit()来完成。

exit（）函数所需的头文件为#include <stdlib.h>，函数原型是：

void exit(int status)

5.2.9 进程的调度

Linux进程调度分为主动调度和被动调度两种方式：

主动调度随时都可以进行，内核里可以通过schedule()启动一次调度，当然也可以将进程状态设置为TASK_INTERRUPTIBLE、TASK_UNINTERRUPTIBLE，暂时放弃运行而进入睡眠，用户空间也可以通过pause()达到同样的目的；如果为这种暂时的睡眠放弃加上时间限制，内核态有schedule_timeout，用户态有nanosleep()用于此目的。

从Linux 2.6内核后，linux实现了***抢占式内核***，

linux2.6内核之前的版本会恢复原进程的运行，直到该进程退出内核态才会引发调度程序；

而linux2.6抢占式内核，在处理完中断后，会立即引发调度，切换到高权值进程。

2. 进程调度的一般原理

调度程序运行时，要在所有可运行的进程中选择最值得运行的进程。选择进程的依据主要有：

进程的调度策略（policy）
静态优先级（priority）
动态优先级（counter）
实时优先级（rt-priority）

Policy是进程的调度策略，用来区分实时进程和普通进程

Counter是实际意义上的进程动态优先级，它是进程剩余的时间片，起始值就是priority的值。

3. Linux O(1)调度

Schedule()函数是完成进程调度的主要函数，并完成进程切换的工作。它在/kernel/sched.c中的定义如下：

```
/*schedule() is the main scheduler function. */
asmlinkage void __sched schedule(void)
{
        struct task_struct *prev，  *next;
        unsigned long *switch_count;
        struct rq *rq;
        int cpu;
need_resched:
        preempt_disable();
        cpu = smp_processor_id();
        rq = cpu_rq(cpu);
        rcu_sched_qs(cpu);
        prev = rq->curr;
        switch_count = &prev->nivcsw;
        release_kernel_lock(prev);
```

schedule的主要工作可以分为两步。
首先是找到next。
1.schedule()检查prev的状态。

2.检查本地运行队列中是否有进程。

3.若本地运行队列中有进程，但没有活动进程队列为空集。这时把活动进程队列改为过期进程队列，把原过期进程队列改为活动进程队列。空集用于接收过期进程。

4.在活动进程队列中搜索一个可运行进程。

5.检查next是否是实时进程以及是否从TASK_INTERRUPTIBLE或TASK_STOPPED状态中被唤醒。

找到next后，就可以实施进程切换了。

1.把next的进程描述符第一部分字段的内容装入硬件高速缓存。
2.清除prev的TIF_NEED_RESCHED的标志。
3.设置prev的进程切换时刻。
4.重新计算并设置prev的平均睡眠时间。
5.如果prev != next，切换prev和next硬件上下文。

这时，CPU已经开始执行next进程了。

主观题 10分

设置

Linux

GNU/Linux 操作系统的基本体系结构?

有哪些管理功能?

进程调度有哪几种方式? 调度原则有?

正常使用主观题需2.0以上版本雨课堂

作答

5.3 Part Three

ARM- Linux内存管理

5.3.1 ARM-Linux内存管理概述

内存管理是Linux内核中最重要的子系统之一，它主要提供对内存资源的访问控制机制。这种机制主要涵盖了：

内存的分配和回收

地址转换

内存扩充

内存的共享与保护

Linux系统会在硬件物理内存和进程所使用的内存（称作虚拟内存）之间建立一种映射关系，这种映射是以进程为单位，因而不同的进程可以使用相同的虚拟内存，而这些相同的虚拟内存，可以映射到不同的物理内存上。

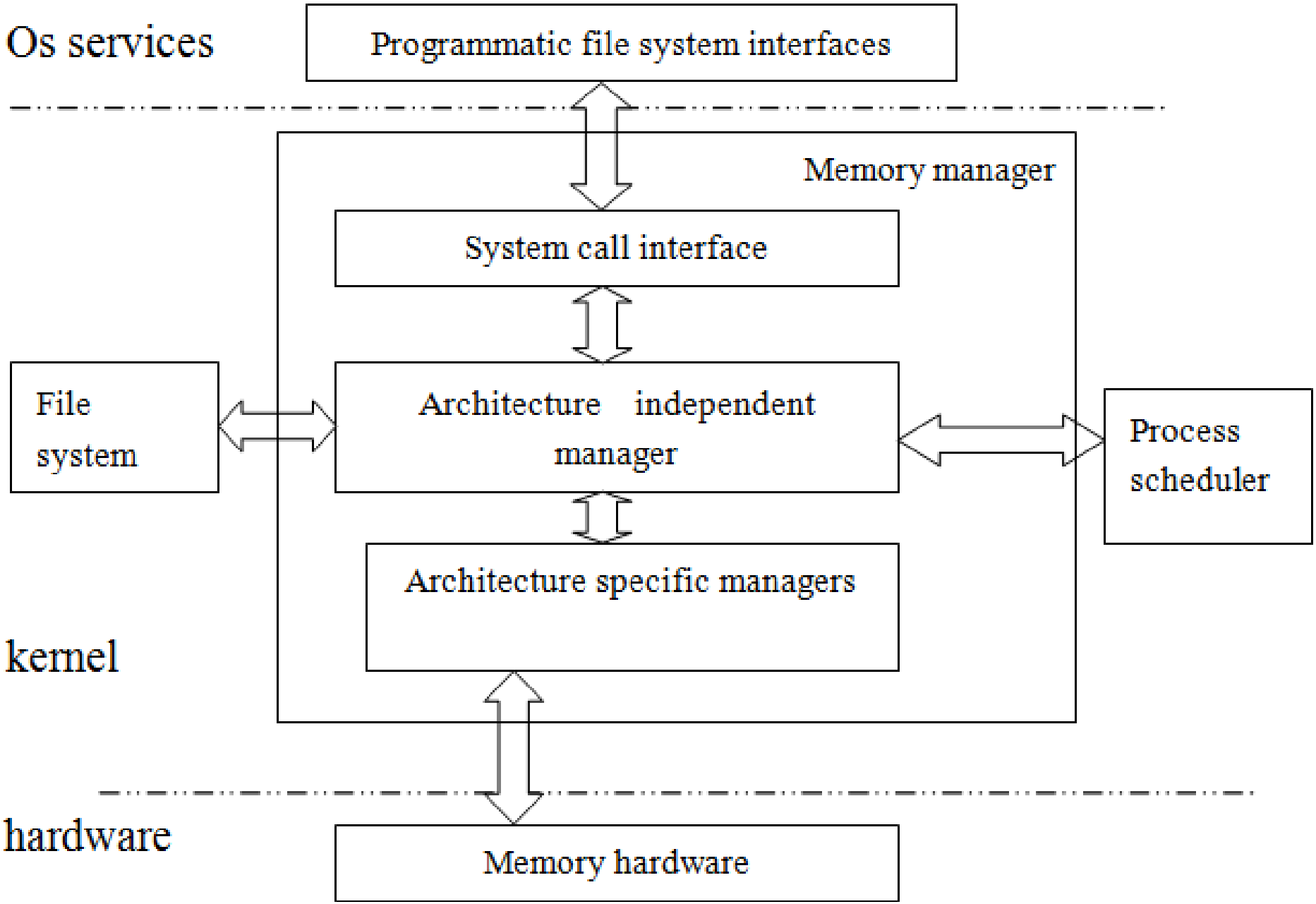


图5-7 内存管理主要子系统架构

Architecture Specific Managers子模块，涉及体系结构相关部分，提供用于访问硬件Memory的虚拟接口。

Architecture Independent Manager子模块，涉及体系结构无关部分，提供所有的内存管理机制，包括以进程为单位的memory mapping、虚拟内存的交换技术Swapping等。

System Call Interface系统调用接口。通过该接口，向用户空间程序应用程序提供内存的分配、释放，文件的映射等功能。

ARM-Linux内核的内存管理功能是采用***请求调页式的虚拟存储技术***实现的。

ARM-Linux内核根据内存的当前使用情况动态换进换出进程页，通过外存上的交换空间存放换出页。

内存与外存之间的相互交换信息是***以页为单位***进行的，这样的管理方法具有良好的灵活性，并具有很高的内存利用率。

5.3.2 ARM- Linux虚拟存储空间及分布

在系统空间，即在内核中，虚拟地址与物理地址在数值上是相同的，至于用户空间的地址映射是动态的，根据需要分配物理内存，并且建立起具体进程的虚拟地址与所分配的物理内存间的映射。

需要值得注意的是，系统空间的一部分不是映射到物理内存，而是映射到一些I/O设备，包括寄存器和一些小块的存储器。

栈：栈是用户存放程序临时创建的局部变量

堆（heap）：堆是用于存放进程运行中被动态分配的内存段，

BSS段：BSS段包含了程序中未初始化的全局变量，

数据段：数据段用来存放可执行文件中已初始化全局变量

代码段：代码段是用来存放可执行文件的操作指令

BSS段：

- BSS段（bss segment）通常是指用来存放程序中未初始化的全局变量的一块内存区域。BSS是英文Block Started by Symbol的简称。BSS段属于静态内存分配。

数据段：

- 数据段（data segment）通常是指用来存放程序中已初始化的全局变量的一块内存区域。数据段属于静态内存分配。

代码段：

- 代码段（code segment/text segment）通常是指用来存放程序执行代码的一块内存区域。这部分区域的大小在程序运行前就已经确定，并且内存区域通常属于只读, 某些架构也允许代码段为可写，即允许修改程序。在代码段中，也有可能包含一些只读的常数变量，例如字符串常量等。

堆（heap）：

- 堆是用于存放进程运行中被动态分配的内存段，它的大小并不固定，可动态扩张或缩减。当进程调用malloc等函数分配内存时，新分配的内存就被动态添加到堆上（堆被扩张）；当利用free等函数释放内存时，被释放的内存从堆中被剔除（堆被缩减）

栈(stack)：

- 栈又称堆栈， 是用户存放程序临时创建的局部变量，也就是说我们函数括弧“{}”中定义的变量。
- 除此以外，在函数被调用时，其参数也会被压入发起调用的进程栈中，并且待到调用结束后，函数的返回值也会被存放回栈中。
- 由于栈的先进先出特点，所以栈特别方便用来保存/恢复调用现场。
- 从这个意义上讲，我们可以把堆栈看成一个寄存、交换临时数据的内存区。

5.3.3 进程空间描述

关键数据结构描述

一个进程的虚拟地址空间主要由两个数据结构来描述。一个是最高层次的：mm_struct，一个是较高层次的：vm_area_structs。

最高层次的mm_struct结构描述了一个进程的整个虚拟地址空间。每个进程只有一个mm_struct结构，在每个进程的task_struct结构中，有一个指向该进程的mm_struct结构的指针，每个进程与用户相关的各种信息都存放在mm_struct结构体中，其中包括本进程的页目录表的地址，本进程的用户区的组成情况等重要信息。

可以说，mm_struct结构是对整个用户空间的描述。

Vm_area_struct是描述进程地址空间的基本管理单元，

Linux内核中对应进程内存区域的数据结构是: vm_area_struct， 内核将每个内存区域作为一个单独的内存对象管理，相应的操作也都一致。

每个进程的用户区是由一组vm_area_struct结构体组成的链表来描述的。用户区的每个段（如代码段、数据段和栈等）都由一个vm_area_struct结构体描述，其中包含了本段的起始虚拟地址和结束虚拟地址，也包含了当发生缺页异常时如何找到本段在外存上的相应内容（如通过nopage函数）。

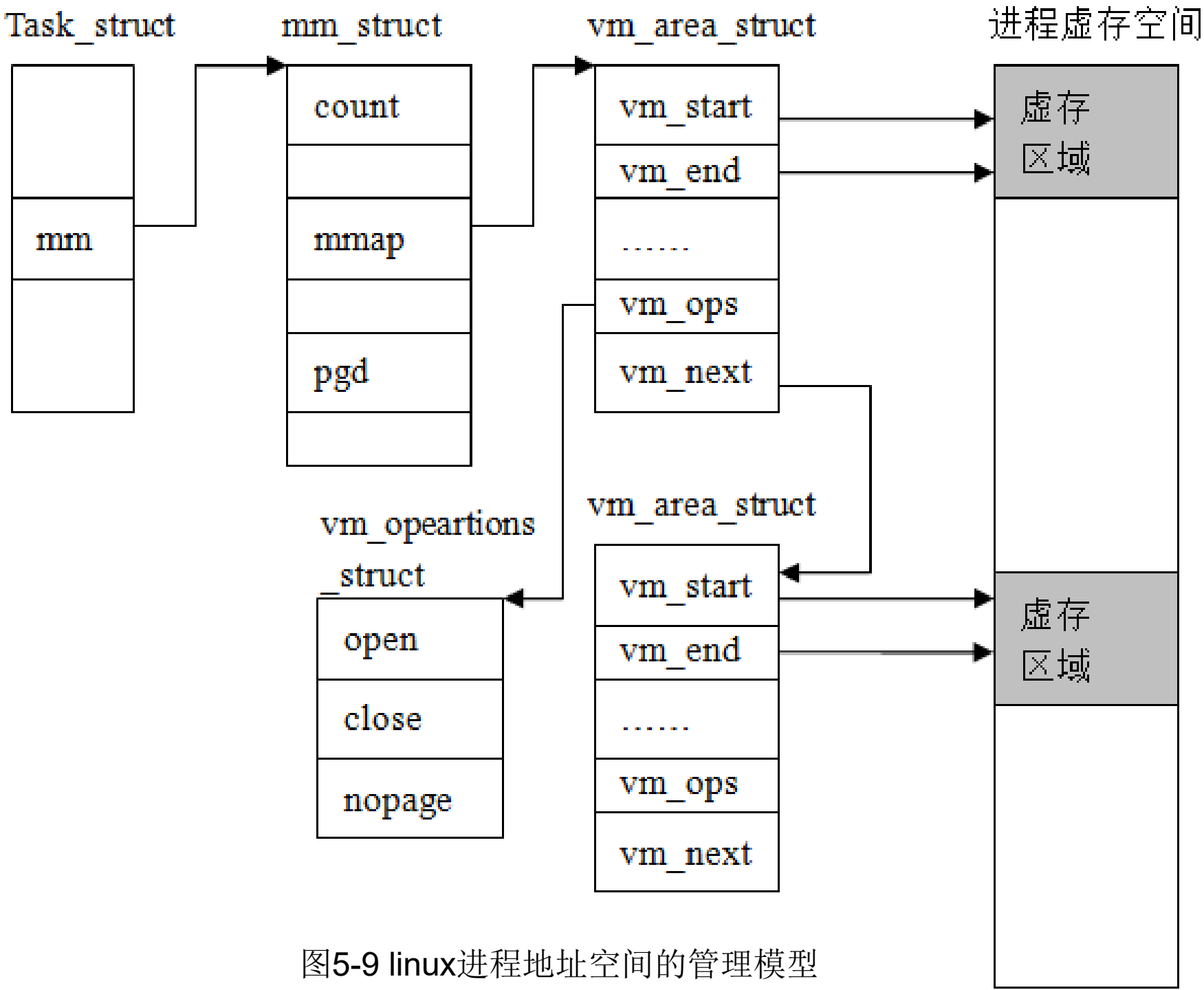
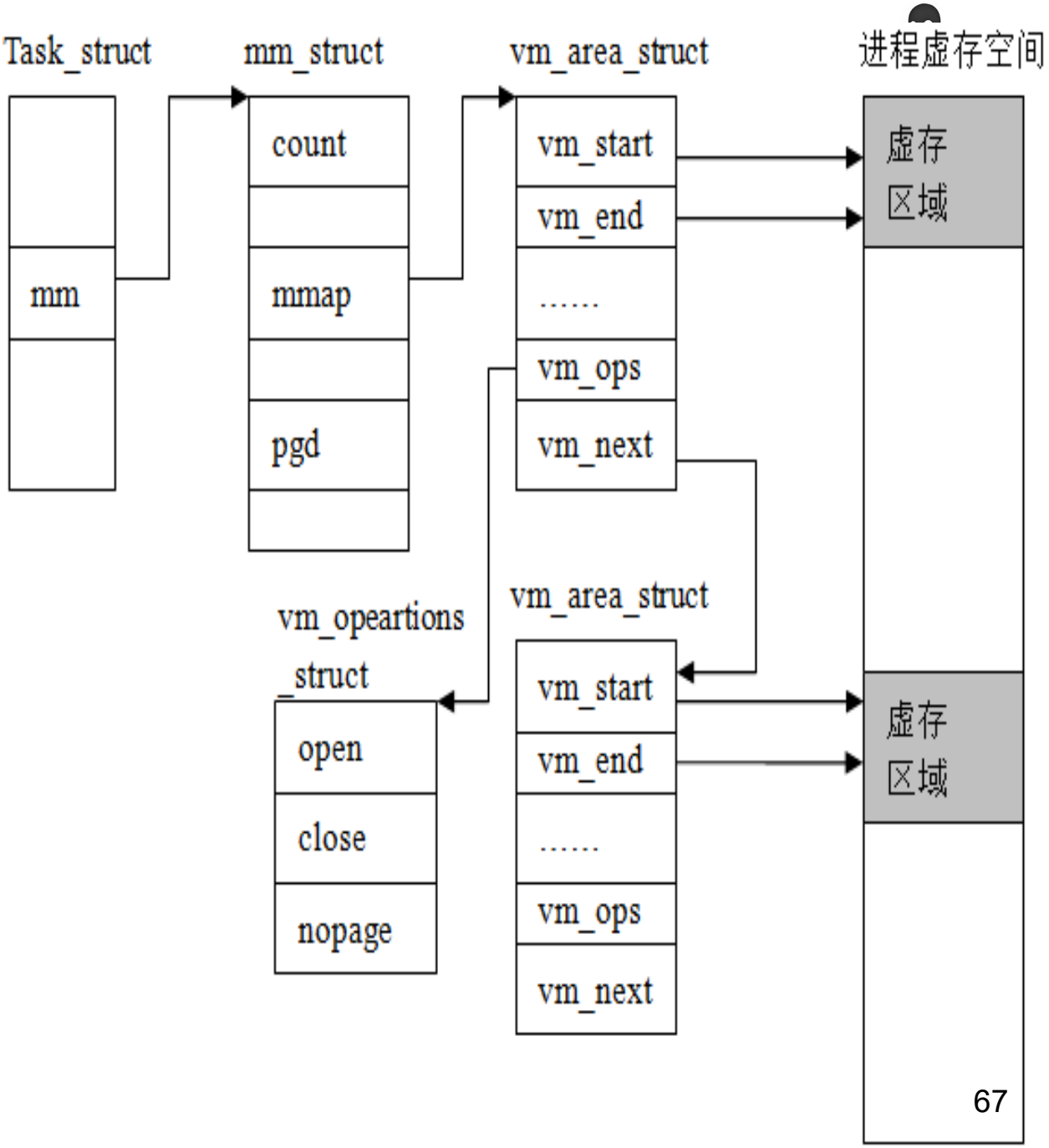


图5-9 linux进程地址空间的管理模型

内存映射(mmap)是Linux操作系统的一个很大特色，它可以将系统内存映射到一个文件（设备）上，以便可以通过访问文件内容来达到访问内存的目的。



vm_area_struct结构体描述如下：

```
struct vm_area_struct {
        struct mm_struct * vm_mm;       /* The address space we belong to. */
        unsigned long vm_start; /* Our start address within vm_mm. */
        unsigned long vm_end;  /* The first byte after our end address
                                          within vm_mm. */
        struct vm_area_struct *vm_next, *vm_prev;
        pgprot_t vm_page_prot; /* Access permissions of this VMA. */
        unsigned long vm_flags; /* Flags, see mm.h. */
        struct rb_node vm_rb;
        union {
                struct {
                        struct list_head list;
                        void *parent;     /* aligns with prio_tree_node parent */
                        struct vm_area_struct *head;
                } vm_set;
                struct raw_prio_tree_node prio_tree_node;
        } shared;
```

2. Linux的分页模型

在Linux2.6中，Linux采用了通用的四级页表结构，四种页表分别称为：页全局目录、页上级目录、页中间目录、页表。

分页机制将整个线性地址空间及整个物理内存看成由许多大小相同的存储块组成的，并把这些块作为**页**（虚拟空间分页后每个单位称为页）或**页帧**（物理内存分页后每个单位称为页帧）进行管理。

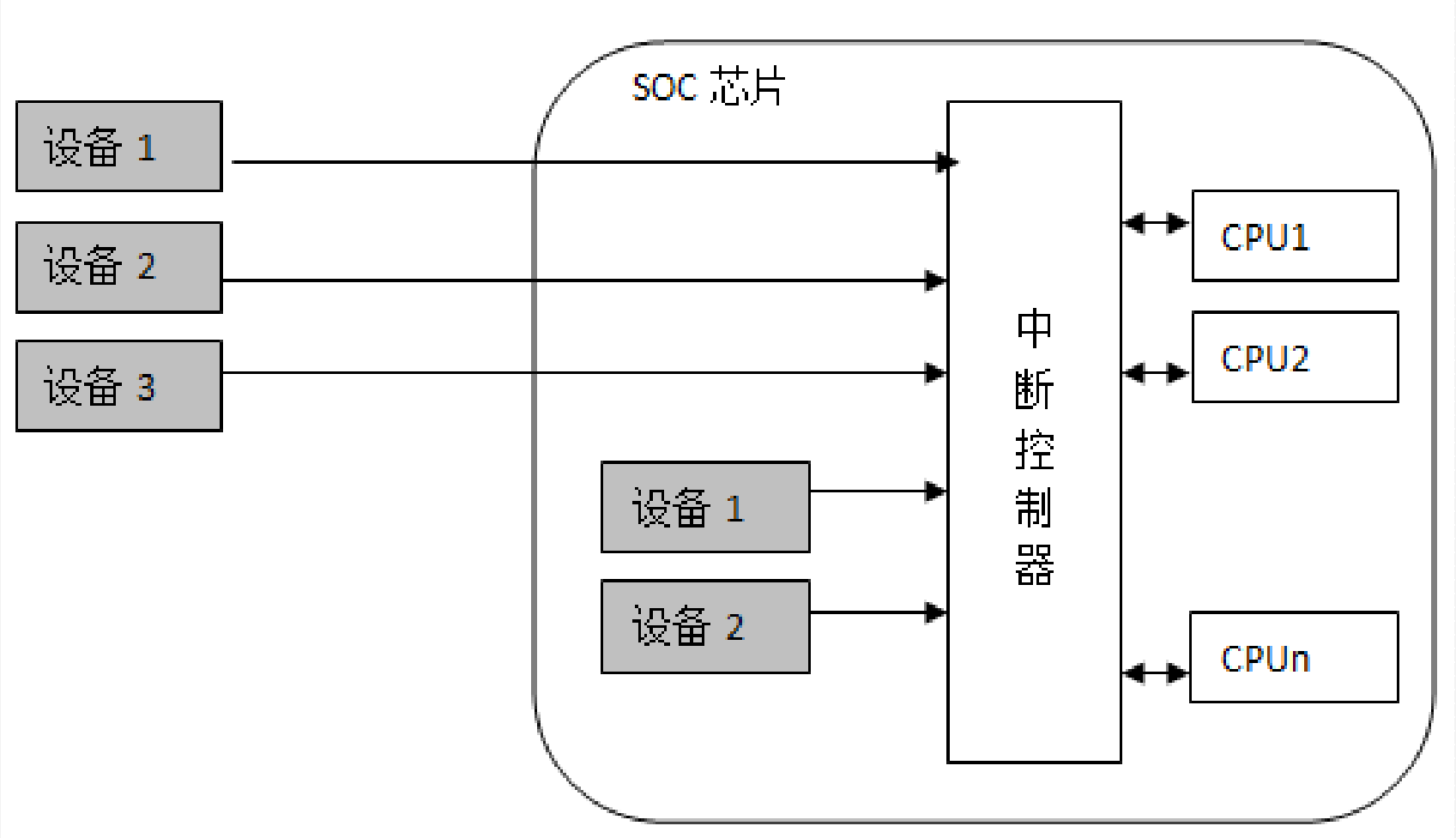
5.5 Part Five

ARM_Linux中断管理

5.5.1 ARM_Linux中断的一些基本概念

1. 设备、中断控制器和CPU

一个完整的设备中，与中断相关的硬件可以划分为3类，它们分别是：设备、中断控制器和CPU本身，



设备： 设备是发起中断的源，当设备需要请求某种服务的时候，它会发起一个硬件中断信号，通常，该信号会连接至中断控制器，由中断控制器做进一步的处理。

中断控制器： 中断控制器负责收集所有中断源发起的中断，ARM架构的soc，使用较多的中断控制器是VIC（Vector Interrupt Controller），进入多核时代以后，GIC（General Interrupt Controller）的应用也开始逐渐变多。

CPU：cpu是最终响应中断的部件，它通过对可编程中断控制器的编程操作，控制和管理者系统中的每个中断。当中断控制器最终判定一个中断可以被处理时，它会根据事先的设定，**通知其中一个或者是某几个cpu对该中断进行处理**

2. IRQ编号

系统中每一个注册的中断源，都会分配一个唯一的编号用于识别该中断，称之为IRQ编号。IRQ编号贯穿在整个Linux的通用中断子系统中。

在移动设备中，每个中断源的IRQ编号都会在arch相关的一些头文件中，例如arch/xxx/mach-xxx/include/irqs.h。

驱动程序在请求中断服务时，它会使用IRQ编号注册该中断，中断发生时，cpu通常会从中断控制器中获取相关信息，然后计算出相应的IRQ编号，然后把该IRQ编号传递到相应的驱动程序中。

5.5.2 内核异常向量表的初始化

ARM-linux 内核启动时，首先运行的是arch/arm/kernel/head.S，进行一些初始化工作，然后调用main.c->start_kernel()函数，进而调用trap_init()（或者调用early_trap_init()函数）以及init_IRQ()函数进行中断初始化，建立异常向量表。

```
asmlinkage void __init start_kernel(void)
{
……
   trap_init();
……
   early_irq_init();
   init_IRQ();
……
}
```

接着系统会建立异常向量表。首先会将ARM处理器异常中断处理程序的入口安装到各自对应的中断向量地址中。

在ARM V4及V4T以后的大部分处理器中，中断向量表的位置可以有两个位置：
一个是0x00000000，另一个是0xffff0000。
V位和中断向量表的对应关系如下：
V=0　～　0x00000000~0x0000001C
V=1　～　0xffff0000~0xffff001C

在采用了**MMU**内存管理单元后，异常向量表放在哪个具体物理地址已经不那么重要了，只需要将它映射到**0xffff0000**的虚拟地址即可。在中断前期处理函数中会根据**IRQ**产生时所处的模式来跳转到不同的中断处理流程中。

Init_IRQ(void)函数是一个特定于体系结构的函数，对于ARM体系结构来说该函数定义如下：

```
void __init init_IRQ(void)
{
        int irq;
        for (irq = 0; irq < NR_IRQS; irq++)
                irq_desc[irq].status |=
IRQ_NOREQUEST | IRQ_NOPROBE;

        init_arch_irq();
}
```

5.5.3 Linux中断处理

1.中断申请并响应

ARM处理器的中断是由处理器内部或者外部的中断源产生，通过**IRQ或者FIQ中断请求**线传递给处理器。在ARM模式下中断可以配成IRQ模式或者FIQ模式。但是在Linux系统里面，所有的中断源都**被配成了IRQ中断模式**。

要想使设备的驱动程序能够产生中断，则首先需要调用**request_irq()来分配中断线**

request_irq()函数的定义：

```
include/linux/interrupt.h
static inline int __must_check
request_irq(unsigned int irq， irq_handler_t handler，
unsigned long flags，
            const char *name， void *dev)
{
        return request_threaded_irq(irq， handler， NULL，
flags， name， dev);
}
```

在通过request_irq()函数注册中断服务程序的时候，将会把设备中断处理程序添加进系统，使在中断发生的时候调用相应的中断处理程序。

request_irq()函数是request_threaded_irq()函数的封装，内核用这个函数来完成分配中断线的工作，

其主要参数说明如下：

irq，要注册的硬件中断号；

handler，向系统注册的中断处理函数，它是一个回调函数

irqflags，中断类型标志，。

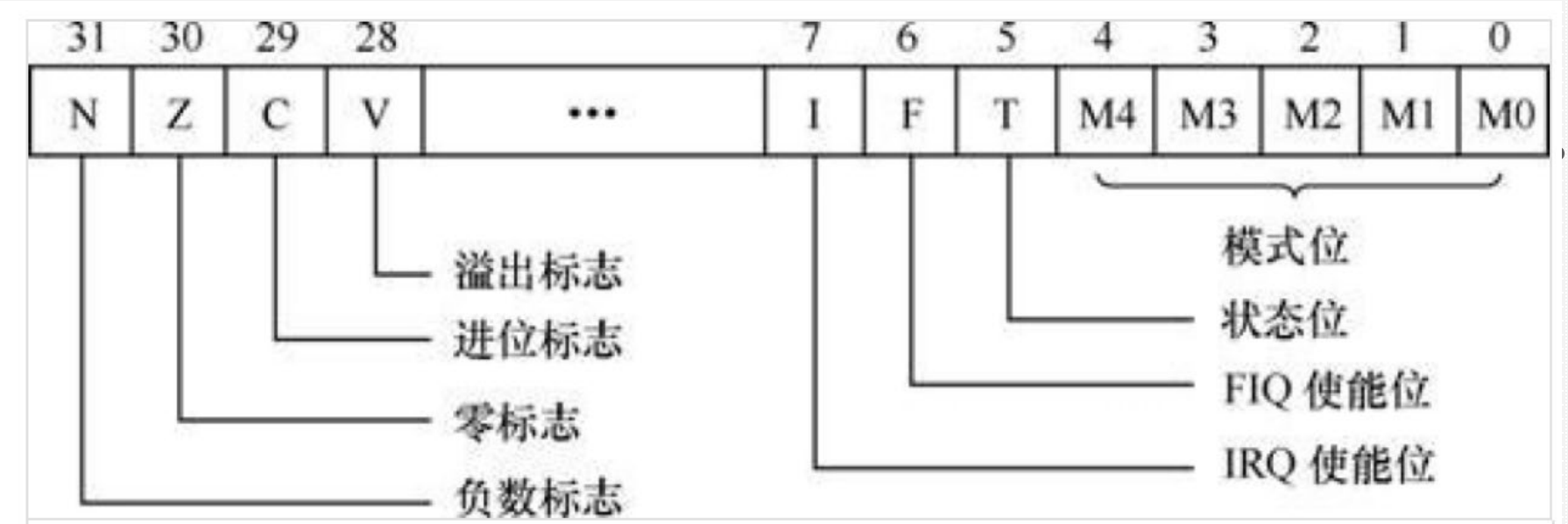
devname，一个声明的设备的ascii 名字，与中断号相关联的名称，在/proc/interrupts文件中可以看到此名称。

dev_id，I/O设备的私有数据字段，thread_fn，由irq handler线程调用的函数，

2.保存现场

处理中断时要保存现场，然后才能处理中断，处理完之后还要把现场状态恢复后才能返回到被中断的地方继续执行。

在指令跳转到中断向量的地方开始执行之前，由CPU自动完成了必要工作之后，每当中断控制器发出产生一个中断请求，则CPU总是到异常向量表的中断向量处取指令来执行。




```
movs  pc，  lr        @ branch to handler in SVC mode
   .long __irq_usr       @  0  (USR_26 / USR_32)
   .long __irq_invalid        @  1  (FIQ_26 / FIQ_32)
   .long __irq_invalid        @  2  (IRQ_26 / IRQ_32)
   .long __irq_svc          @  3  (SVC_26 / SVC_32)
```

3.中断处理

ARM Linux对中断的处理主要分为**内核模式下的中断处理模式**和**用户模式下的中断处理模式**。

内核模式下的中断处理，也就是调用__irq_svc例程，__irq_svc例程在文件arch/arm/kernel/entry-armv.S中定义，首先来看这个例程的定义：

```
__irq_svc:
   svc_entry

#ifdef CONFIG_PREEMPT
   get_thread_info tsk
   ldr   r8，  [tsk，  #TI_PREEMPT]      @ get preempt count
   add   r7，  r8，  #1          @ increment it
   str   r7，  [tsk，  #TI_PREEMPT]
#endif
```

irq_handler

4.中断返回

中断返回在前文已经分析过，这里不再赘述。这里只补充说明一点：如果是从用户态中断进入的则先检查是否需要调度，然后返回。如果是从系统态中断进入的则直接返回。

5.6 Part Six

ARM-Linux系统调用

Linux系统利用SWI指令来从用户空间进入内核空间。SWI指令用于产生软件中断，从而实现从用户模式到管理模式的变换，CPSR保存到管理模式的SPSR，执行转移到SWI向量。在其他模式下也可使用SWI指令，处理器同样地切换到管理模式。指令格式如下：

SWI {cond} immed_24

Cond域：是可选的条件码，. immed_24域：范围从 0 到 $2^{24}-1$的表达式，（即0-16777215），immed_24为软中断号(服务类型)。

使用SWI指令时，通常使用以下两种方法进行传递参数，如：

(1) 指令中的24位立即数指定了用户请求的服务类型，参数通过通用寄存器传递.

```
mov  r0，#34   ;设置子功能号位34
SWI   12       ;调用12号软中断
```

(2) 指令中的24位立即数被忽略，用户请求的服务类型由寄存器R0的值决定，参数通过其他的通用寄存器传递．如：

```
mov r0，#12     ;调用12号软中断
mov r1，#34     ;设置子功能号位34
SWI       0
```

深入理解系统调用

本章小结：